Sistema de Gestión de Agua Potable

Documentación de Diagramas

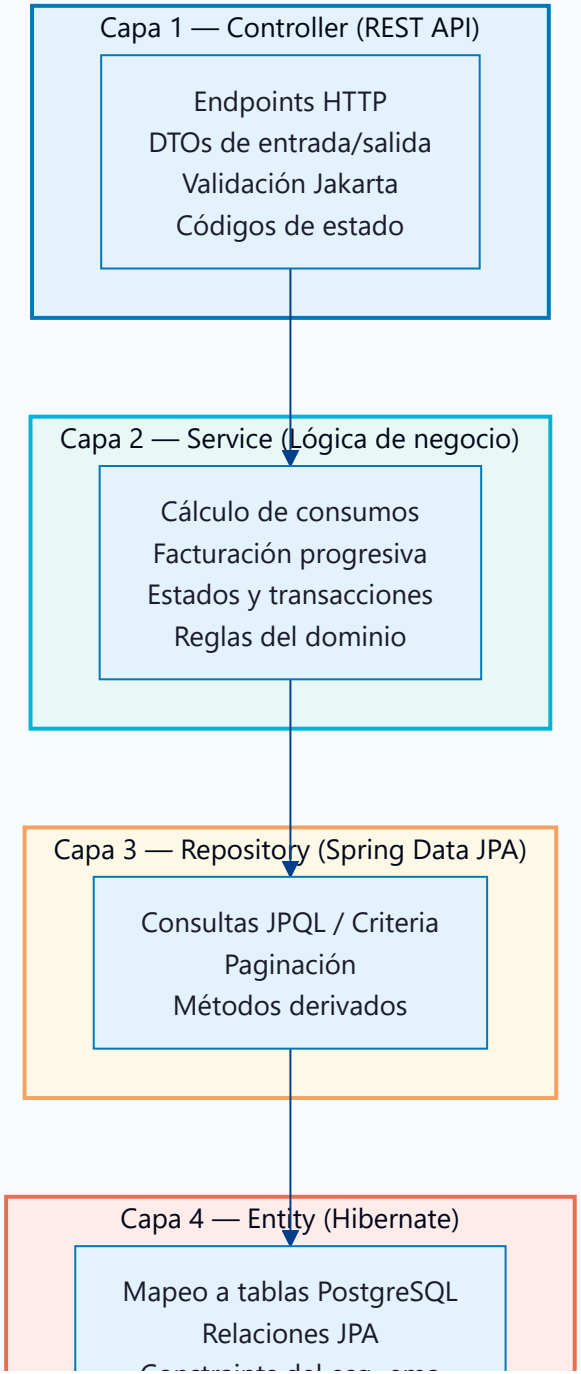
1. Arquitectura por capas
2. Diagrama de clases / paquetes
3. Diagrama Entidad–Relación (ER)
4. Diagrama de secuencia: Emisión de factura
5. Diagrama de secuencia: Registro de lectura y pago
6. Diagrama de despliegue
7. Mapa de módulos y endpoints

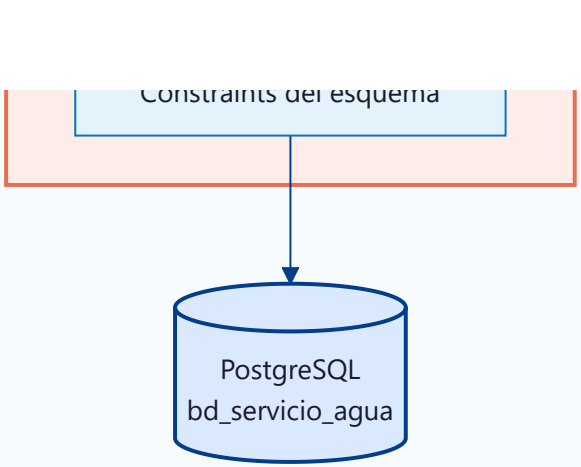
Java · Spring Boot · Spring Data JPA · PostgreSQL · Maven

Arquitectura por capas · Septiembre 2026

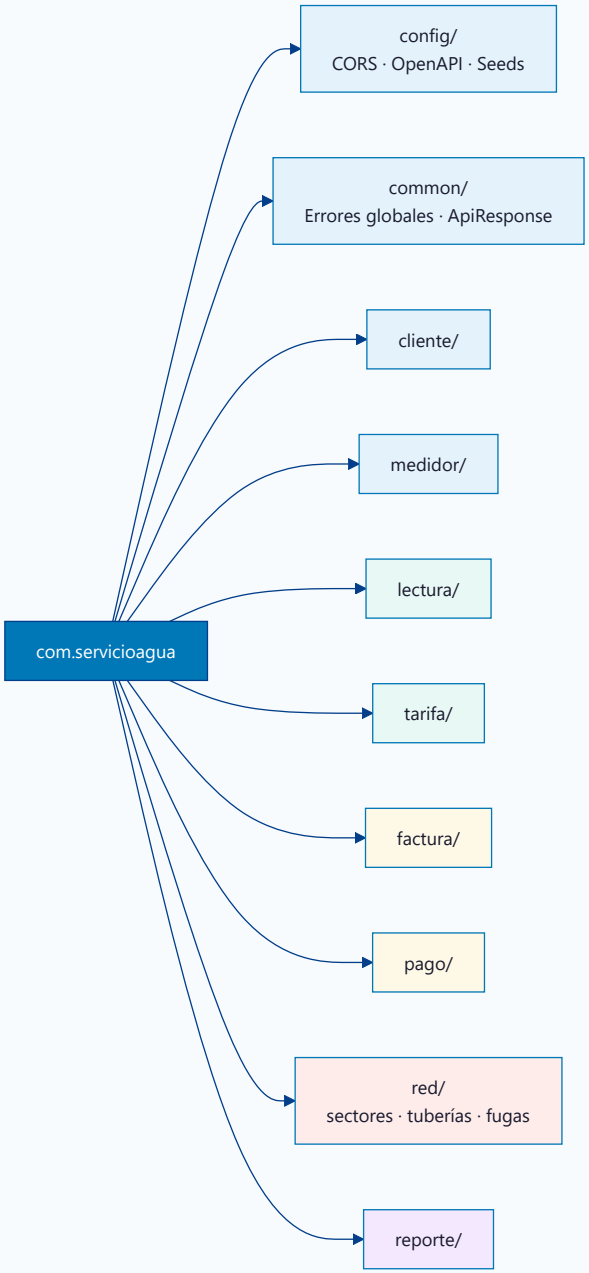
1. Arquitectura por capas

Separación de responsabilidades en 4 capas: Controller → Service → Repository → Entity.


Capa 1 — Controller (REST API)
Endpoints HTTP
DTOs de entrada/salida
Validación Jakarta
Códigos de estado
Capa 2 — Service (Lógica de negocio)
Cálculo de consumos
Facturación progresiva
Estados y transacciones
Reglas del dominio
Capa 3 — Repository (Spring Data JPA)
Consultas JPQL / Criteria
Paginación
Métodos derivados
Capa 4 — Entity (Hibernate)
Mapeo a tablas PostgreSQL
Relaciones JPA



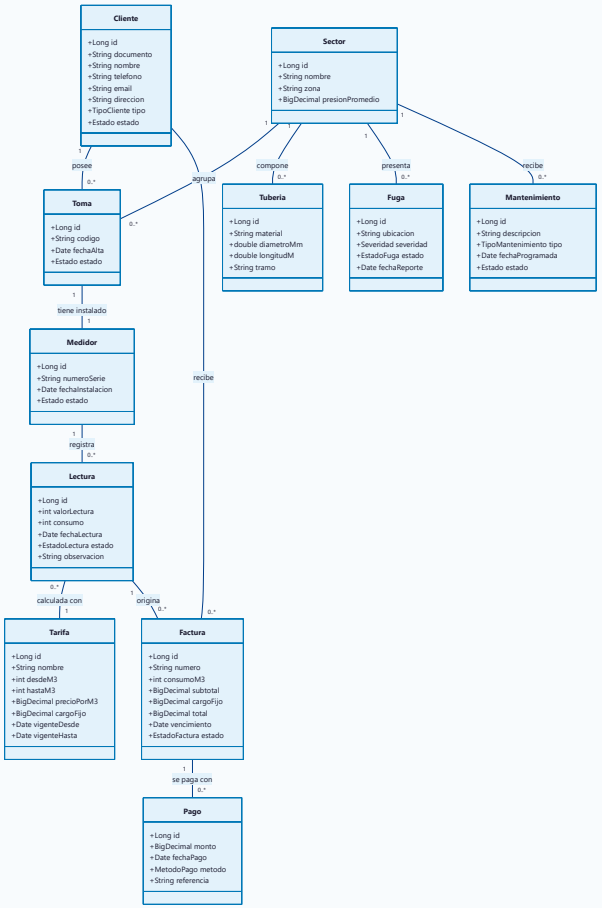
Estructura de paquetes


com.servicioagua
config/
CORS · OpenAPI · Seeds
common/
Errores globales · ApiResponse
cliente/
medidor/
lectura/
tarifa/
factura/
pago/
red/
sectores · tuberías · fugas
reporte/

Cada módulo sigue la misma estructura interna: **controller / service / repository / entity / dto**.

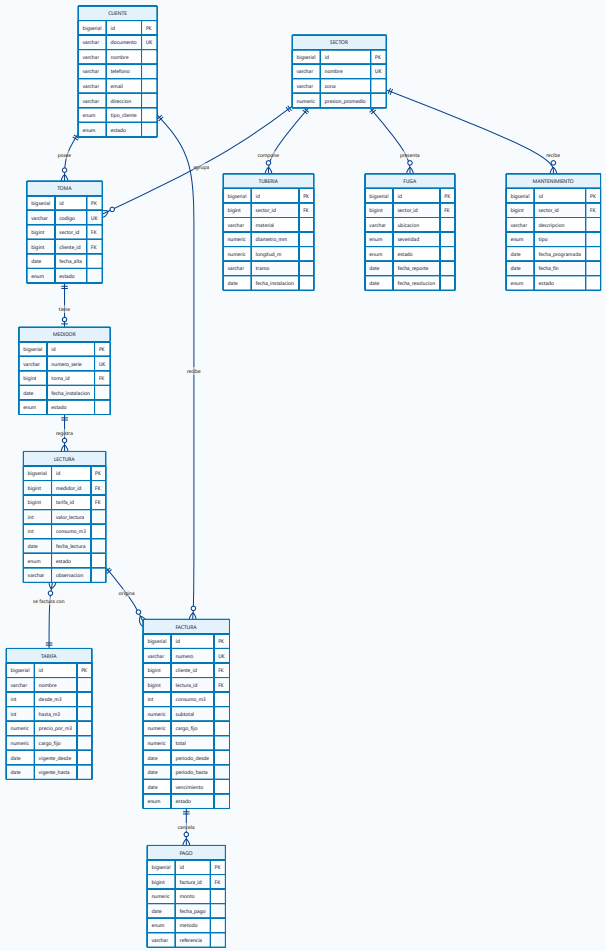
2. Diagrama de clases (dominio principal)

Clases JPA con atributos y multiplicidades entre los módulos del negocio.


Cliente
+Long id
+String documento
+String nombre
+String telefono
+String email
+String direccion
+TipoCliente tipo
+Estado estado
Sector
+Long id
+String nombre
+String zona
+BigDecimal presionPromedio
posee
agrupa
compone
presenta
recibe
Toma
+Long id
+String codigo
+Date fechaAlta
+Estado estado
Tuberia
+Long id
+String material
+double diametroMm
+double longitudM
+String tramo
Fuga
+Long id
+String ubicacion
+Severidad severidad
+EstadoFuga estado
+Date fechaReporte
Mantenimiento
+Long id
+String descripcion
+TipoMantenimiento tipo
+Date fechaProgramada
+Estado estado
tiene instalado
Medidor
+Long id
+String numeroSerie
+Date fechaInstalacion
+Estado estado
registra
Lectura
+Long id
+int valorLectura
+int consumo
+Date fechaLectura
+EstadoLectura estado
+String observacion
calculada con
origina
Tarifa
+Long id
+String nombre
+int desdeM3
+int hastaM3
+BigDecimal precioPorM3
+BigDecimal cargoFijo
+Date vigenteDesde
+Date vigenteHasta
Factura
+Long id
+String numero
+int consumoM3
+BigDecimal subtotal
+BigDecimal cargoFijo
+BigDecimal total
+Date vencimiento
+EstadoFactura estado
se paga con
Pago
+Long id
+BigDecimal monto
+Date fechaPago
+MetodoPago metodo
+String referencia

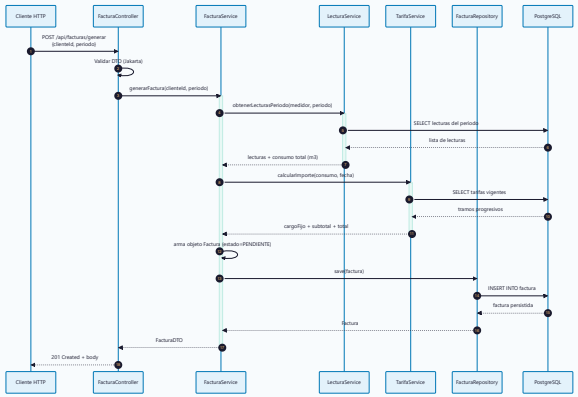
3. Diagrama Entidad–Relación (ER)

Modelo lógico de la base de datos PostgreSQL. PK = clave primaria, FK = clave foránea, UK = única.


CLIENTE
bigserial id PK
varchar documento UK
varchar nombre
varchar telefono
varchar email
varchar direccion
enum tipo_cliente
enum estado
SECTOR
bigserial id PK
varchar nombre UK
varchar zona
numeric presion_promedio
posee
abarca
compuesta
presenta
recibe
TOMA
bigserial id PK
varchar codigo UK
bigint sector_id FK
bigint cliente_id FK
date fecha_alta
enum estado
TUBERIA
bigserial id PK
bigint sector_id FK
varchar material
numeric diametro_mm
numeric longitud_m
varchar tramo
date fecha_instalacion
FUGA
bigserial id PK
bigint sector_id FK
varchar ubicacion
enum severidad
enum estado
date fecha_reporte
date fecha_resolucion
MANTENIMIENTO
bigserial id PK
bigint sector_id FK
varchar descripcion
enum tipo
date fecha_programada
date fecha_fin
enum estado
tiene
MEDIDOR
bigserial id PK
varchar numero_serie UK
bigint toma_id FK
date fecha_instalacion
enum estado
registra
LECTURA
bigserial id PK
bigint medidor_id FK
bigint tarifa_id FK
int valor_lectura
int consumo_m3
date fecha_lectura
enum estado
varchar observacion
se factura con
origina
FACTURA
bigserial id PK
varchar numero UK
bigint cliente_id FK
bigint lectura_id FK
int consumo_m3
numeric subtotal
numeric cargo_fijo
numeric total
date periodo_desde
date periodo_hasta
date vencimiento
enum estado
TARIFA
bigserial id PK
varchar nombre
int desde_m3
int hasta_m3
numeric precio_por_m3
numeric cargo_fijo
date vigente_desde
date vigente_hasta
cancela
PAGO
bigserial id PK
bigint factura_id FK
numeric monto
date fecha_pago
enum metodo
varchar referencia

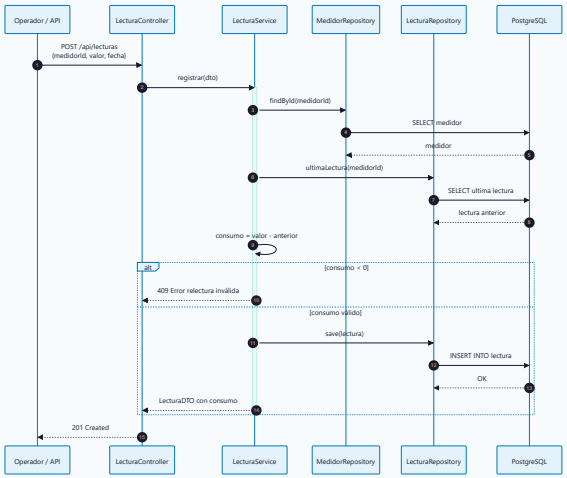
4. Diagrama de secuencia — Emisión de factura

Flujo de generación de factura: lee consumos del período, aplica tarifas progresivas y persiste el documento.

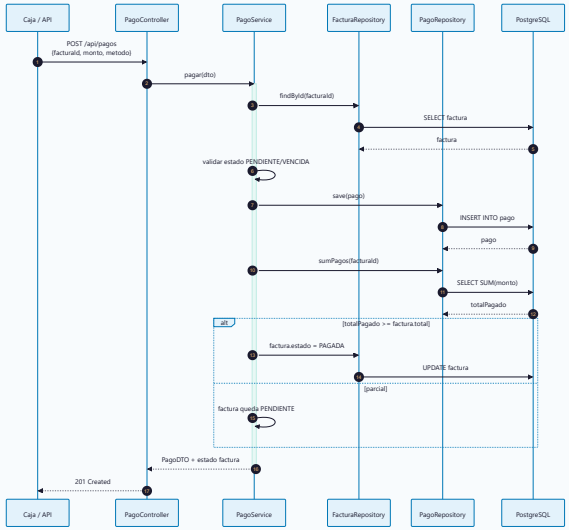


5. Diagramas de secuencia — Lectura y Pago

5.1 Registro de lectura con cálculo de consumo

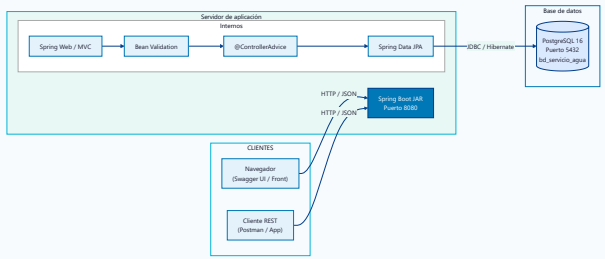


5.2 Registro de pago y actualización de factura


Caja / API
PagoController
PagoService
FacturaRepository
PagoRepository
PostgreSQL
POST /api/pagos
(facturaId, monto, metodo)
pagar(dto)
findById(facturaId)
SELECT factura
factura
validar estado PENDIENTE/VENCIDA
save(pago)
INSERT INTO pago
pago
sumPagos(facturaId)
SELECT SUM(monto)
totalPagado
alt
[totalPagado >= factura.total]
factura.estado = PAGADA
UPDATE factura
[parcial]
factura queda PENDIENTE
PagoDTO + estado factura
201 Created
Caja / API
PagoController
PagoService
FacturaRepository
PagoRepository
PostgreSQL

6. Diagrama de despliegue

Arquitectura de ejecución: aplicación Spring Boot empaquetada como JAR, conectada a PostgreSQL.



7. Mapa de módulos y endpoints

Vista general de la API REST bajo el prefijo `/api`.



Reglas de negocio clave

#	Regla
1	Consumo = lectura actual − lectura anterior; si el resultado es negativo, se rechaza (relectura / anulación).
2	Facturación progresiva: el consumo se reparte por tramos de tarifa vigente a la fecha del período.
3	Solo un medidor activo por toma; al cambiar medidor se cierra el anterior con lectura de cierre.
4	Factura queda PAGADA cuando SUM(pagos) ≥ total; admite pagos parciales.
5	Facturas con vencimiento superado y sin pago total pasan a estado VENCIDA.
6	Una toma se da de baja solo si su medidor no tiene lecturas pendientes del período.